

Covertypes District Rollout

Tensium Samples

Pack identification

Category

CPU ML engineering

Pack / category name

Covertypes District Rollout

Why current models fail, and the capability this task builds

The informative failures find the real defect, dropping the location-identity features and reweighting the classes, yet stop one level short: with the target district withheld and validation reduced to a rate-limited clean/dirty bit, they calibrate on proxies built from the visible data, and the promoted configuration lands a fraction under the recall bar on the second dominant class. The capability isolated is distribution-shift auditing: recognizing that a pipeline which looks optimal on every visible metric is leaning on population-identity features that cannot transfer, and correcting it without direct access to the population it will serve. This is the exact defect that ships green offline and fails in the field when a model meets a new region, so a model that learns to catch it gains the ability to audit its own generalization before it deploys.

1. Description

Description

CPU-only ML incidents where a model that looks healthy on every metric the team can see fails on the population it is being rolled out to. The agent inherits a working pipeline and a failing rollout, and must find the cause by reading the data rather than by tuning: the shipped configuration is the visible optimum, so anything optimised against the agent's own holdout moves away from the answer. It then drives the change through a gated release: deploy, validate against a deliberately lossy signal, diagnose and recover a forced serving failure, canary, cut over, submit. Buyers get a task where the failure mode is a confident, complete, wrong answer produced by a defensible method, which is the failure worth training against. Step-exhaustion and crashes are designed against rather than eliminated: the step budget is generous and the deliverable is written early and overwritten. One measured rollout still ended early; section 10 reports exactly what it did rather than hiding it, and counts it as a failure.

2. Task type & unit type

Task type

Hands-on long-horizon CPU ML engineering: observe -> reproduce -> edit -> deploy -> validate -> diagnose -> recover -> revalidate -> promote -> canary -> cutover -> submit.

Unit type

One self-contained release incident: its own real dataset, broken-but-plausible pipeline, root-owned gated operations daemon, held-out grading population, reference solution, verifier, control battery and frozen terminal.

3. Verification method

Verification method

Graded by value, environment-owned. The daemon scores the promoted artefact itself against a held-out district the agent cannot read, and freezes the result. The agent's code is never imported into the grader: it runs as an unprivileged subprocess with a timeout, receives features only, and its stdout is never trusted. The grader then replays the root-owned action log and compares the frozen terminal by value against a salted hash. Full reward requires a conjunction of terminal correct, the ordered ten-step gate walk, the forced recovery, and the depth floors, and only that pays 1.0. Below it a dense ordered ladder separates partial progress, because an earlier build scored every failure 0.000 and two model tiers came out indistinguishable. Four structurally different correct implementations all score 1.0, which is what shows grading is by value rather than by source shape.

4. Technical domains / adjacent benchmarks

Technical domains

Classical CPU ML (scikit-learn), distribution shift and covariate drift, leakage via identity features, class-imbalance handling, train/serve consistency, feature-spec versioning and stale caches, gated release engineering, Unix privilege separation, and deterministic verifier design.

Adjacent benchmarks

Terminal-Bench-like long-horizon software engineering; tau-bench-like stateful tool use over a gated operation surface; agentic ML-debugging benchmarks.

5. Human experts used

Human experts

The task was authored and calibrated. Independent review and QA feedback was supplied by the Tensium task-review team. No additional credential claims are made.

6. Raw base data

Raw base data

UCI Forest Covertypes, 581,012 real 30m cells from Roosevelt National Forest, sha256 614360d0...b15771.

The generators SELECT and LAY OUT real rows; they never mint a value. Every number the agent sees and every number the grader checks is a measurement of the source file, and the raw archive ships in the review zip so the reproduction actually runs.

The discriminating signal is a genuine property of the data, not an arrangement. The held-out district is a fourth wilderness area the training table never contains, so its wilderness-area one-hot is district

identity with no transferable signal, and its cover-type mix is close to inverted relative to training. Its soil codes also barely overlap the training districts (Jaccard 0.20), but that feature is corroborating rather than load-bearing: the fix does not require dropping it. This was verified against the published source rather than assumed; the generator asserts the property and fails the build if the data stops satisfying it.

7. What the world looks like

What the world looks like

/workspace/target inside a CPU-only Docker image, no internet, running as uid 1000. The agent sees the full pipeline source, its config, the training table, a schema and a short domain brief, and drives the release through a typed env_cli.py surface. A root-owned daemon on a Unix socket holds everything it must not see (the held-out population, the pass bars, the append-only action log and the frozen terminal) under root:envdata 0750. The socket, not setuid, is the privilege boundary, because the served sandbox sets no_new_privs and a sudo-based path is unreachable there. Observation ops are free, unlimited and logged; validation is rate-limited and returns one bit.

8. Time per task

Time per task

8 - 10 hours of authoring and QA per task for the build itself.

9. QA process

QA process

QA runs through a systematic, automated pipeline rather than an ad-hoc pass. Deterministic controls mechanically screen the task for the failure modes that matter, reward-hacking and forgeable graders, answer leakage and reachable oracles, in-process grading, and false difficulty (for example, a "failure" that was only budget truncation). Each control is two-condition validated: it must fire on a planted defect and stay silent on a clean task. An adversarial control battery then executes cheat submissions against the real grader and requires every one to score zero, while a gold solution and a structurally different correct solution both score 1.0. An independent review pass grounds every finding in the actual task files, and difficulty is calibrated by running a frontier model at a fixed sampling standard inside the real isolated environment, with guards that separate genuine failures from budget/truncation artifacts.

QA includes an oracle solution, an alternate structurally different implementation, two independent reference engines, deterministic hidden scenarios, subprocess execution, isolation checks, timeout protection, and an anti-leak Docker build guard. Gate 0 successfully produced a 1.0 reward, confirming the reference solution can complete the task correctly. Full negative-control evidence should only be marked complete when the corresponding scored runs are available.

10. Difficulty calibration

SOTA model & pass@1

Claude Opus 5, 4 independent runs on the shipped build: 1/4 strict passes (25%). Run 3 scored 1.00: the full gated trajectory, with both recall bars and the balanced-accuracy skill gate cleared. Runs 1 and 4 scored 0.67: each drove the entire release and cleared the injected fault but landed a fraction

under the second recall bar. Run 2 scored 0.37: it validated clean but reached the 100 tool-call step budget before completing the release. No infrastructure failures.

Rollout records

covertime-district-rollout-traces.zip (per-rollout action logs and graded JSON). Frozen task package (covertime-district-rollout.zip) SHA-256:
4b4f9346788d2984c8a331cd5343ca3d18a387bb1f007133b9fbe81820c4f561.

11. Grading-design table

Component	What it measures	Weight / cap	How it is enforced
started	Baseline recorded and a candidate deployed	0.05	Accepted reproduce + deploy in the root-owned ledger
gate walk	The full 10-step ordered release sequence	0.10	Action ledger replay against the required order
forced recovery	Injected serving fault diagnosed and cleared with the matching family	0.20	Fault fires -> diagnostics -> matching recovery; wrong family costs a validation
depth	The run actually investigated	0.05	15 accepted actions, 4 observations, 3 distinct kinds; logged daemon-side
moved_a_class	Model genuinely improved over the shipped baseline	0.15	Root-side score vs the held-out population, best-so-far at each validation
bars_cleared	Each graded class bar cleared	0.12 per class	Per-class recall vs bars.json, computed root-side on hidden labels
skill gate	Promoted model genuinely classifies the district, not just the two bars	hard (in terminal)	Balanced accuracy ≥ 0.27 ; a feature-blind policy sits at 0.25 and fails
PARTIAL CAP	Ceiling on any non-passing run	0.70	No conjunction-failing run can reach 1.0
strict_pass	Full conjunction	1.00	Boolean conjunction of all four; terminal by salted hash

12. How the tasks were created

How created

The task was built from a real property of the UCI Covertime data rather than an injected fault: the held-out district is a fourth wilderness area absent from the training table, so its wilderness-area one-hot carries district identity rather than transferable signal, and its cover-type mix is close to inverted; the genuine fix drops that location one-hot and applies bounded class reweighting, while

the soil codes (whose Jaccard overlap with training is 0.20) are corroborating and optional to drop. Substrate candidates were screened by measurement before any content was written, and accepted only where a 2x2 over the candidate edits showed the visible-metric maximum landing on a failing cell, so the observable signal actively points at a wrong configuration rather than merely failing to help; candidates that did not discriminate were kept as measurements in the review bundle rather than as assertions. The bars were set the same way, from a measured sweep of the shipped pipeline, so exactly the genuine fix clears them and over-correction fails the second bar, making the answer a bounded region rather than a direction to push in. The oracle drives the complete trajectory as the positive control, and nineteen negative controls covering the familiar-but-wrong fix, the decoy fix, over-correction, one-shot submission, forged logs and state, symlink exfiltration through the privileged deployment copy, held-out label reads, skipped diagnosis, wrong recovery family and a feature-blind label-spamming policy are graded on every gate run. The deciding signal is forced through operations, because the graded population is never in the agent's tree and the only feedback is a rate-limited one-bit validation; submit sits behind cutover, canary, promote and a post-recovery validation, and the one-shot control scores 0.000.

13. QA checklist

- ☒ Two-condition proof: positive control (oracle ~1.0) AND negative controls (wrong work scores low, for the intended reason). Gold 1.000 deterministic; 19 named negatives, each flooring for its stated reason.
- ☒ For hands-on tasks: a SECOND, structurally different correct solution also scores ~1.0. `pc_fix_first`, `pc_recover_first`, `pc_iterate_after_recovery` and `pc_family_bruteforce` all reach 1.000 via different routes.
- ☒ Grader isolation: grader, answers and reference solution live outside `/workspace/target` (in `tests/` and `_private/`).
- ☒ For hands-on tasks: agent code runs as a subprocess with a timeout, never imported into the grader. It receives features only and its stdout is not trusted.
- ☒ No answer / solution leakage: nothing in environment/workspace reveals the answer; Dockerfile build guard fails the build on held-out data or a planted reference.
- ☒ Deducibility: everything needed is in the workspace; the task is solvable; no invisible coin-flips.
- ☒ Real data, not synthetic: UCI Forest Covertypes; the generator selects real rows and mints nothing; the raw archive ships in the review zip.
- ☒ Long-horizon by design: deciding signal forced, results redirect the next step, no rewarded elective depth; one-shot control scores 0.000.
- ☒ Build hygiene: zip staged in a temp dir; `_private/` excluded from the client zip; verified on the shipped artefact.
- ☒ Scrubbed: no secrets or keys, no platform job-ids or URLs, no internal process jargon or model ids in shipped files.

14. Package overview & contents

This submission covers one task package in detail. The package is self-contained and includes the agent instructions, editable workspace, container environment, hidden verifier assets, deterministic grader and reference solution.

Field	Submission detail
Selected package	covertime-district-rollout.zip
Task count	1 self-contained Harbor task
Task	cpu-ml-engineering/covertime-district-rollout
Task type	CPU ML engineering; hands-on long-horizon investigation, repair, deployment and recovery
Unit / deliverable	One isolated task instance; corrected deployed pipeline plus clean validation, forced-fault recovery, promotion and submission
Verification	Environment-owned terminal graded by value (two class-recall bars plus a balanced-accuracy skill gate), an ordered gate walk, forced stale-cache recovery, a depth floor and isolated execution
Raw base data	UCI Forest Cvertime (public)

15. Per-task deep-dive

The task-level inventory for the included sample package.

Task	What the task is about	Deliverable
Covertime District Rollout	A forest-cover pipeline passes every visible check yet fails when rolled out to a fourth district, because it leans on location-identity features and the district's class mix is close to inverted. Diagnose the cause from gated evidence, repair the pipeline, and drive the gated release.	A deployed pipeline whose promoted candidate clears both dominant-class recall bars and the balanced-accuracy skill gate on the held-out district, reached through clean validation, forced-fault recovery, promotion and successful submission.

16. Example prompt

The agent-facing prompt (instruction.md).

we extended the cover-type model to a fourth district last month and it is not working there. foresters are getting cover-type calls that are wrong often enough that they have stopped trusting the tool, and the crew that owns the model has moved on, so this has landed with you. the model itself looks fine from here, which is the confusing part. it retrains cleanly, the unit tests pass, and its holdout accuracy is the best it has ever been. whatever is wrong is not visible from the numbers we have in the repo.

everything is in /workspace/target/. start with `python3 env\_cli.py inspect` and `python3 scripts/train.py` to see where you are, then work out the fix from the data. drive it through the release path with `python3 env\_cli.py`: reproduce -> deploy -> validate -> diagnostics -> recovery <family> -> validate -> promote -> canary_start -> canary_read -> cutover_commit -> submit. validation is rate-limited and only tells you clean or dirty, and at some point serving will throw a fault you must diagnose and recover before it comes back clean. grading looks at the deployed code and the driven trajectory, not anything left in the workspace: the district's cover is dominated by types 3 and 6, and the promoted model must clear at least 44% recall on each plus a balanced-accuracy skill gate on that district.

17. Environment & execution

Requirement	Specification
Runtime	Python 3.12-slim container; unprivileged ubuntu agent workspace (uid 1000)
Dependencies	numpy 2.1.3; scipy 1.14.1; scikit-learn 1.5.2; pandas 2.2.3; PyYAML 6.0.2; joblib 1.4.2; pytest 8.3.3
Compute	4 CPUs; 8,192 MB memory; 10,240 MB storage; 0 GPUs
Time limits	Agent 7,200 s; verifier 1,200 s; environment build 3,600 s
Visible tests	python3 -m pytest tests/
Baseline	python3 scripts/train.py
Operations	python3 env_cli.py <op>, followed by the required ordered gate sequence
Verification	tests/test.sh executes python3 /tests/grade.py
Network policy	Internet access disabled

18. Human, AI & automation roles

Contributor	Role in design and production
Human experts	Selected and analyzed the public UCI Covertypes data; identified the genuine distribution-shift signal; authored the environment, oracle and negative controls; performed privilege-separation hardening; reviewed failures and approved the frozen task. Independent review by the Tensium task-review team identified and closed a recall-only grading weakness (balanced-accuracy skill gate added).
AI systems	Four Claude Opus 5 runs measured difficulty on the shipped build; a lower-tier model (DeepSeek V4 Flash) was run for separation; behaviorally different passing routes confirmed that grading is by value, not by one source-code shape.
Automation	Container builds, package leak guards, the negative-control battery, gate scripts, the salted terminal hash, root-owned action-ledger checks, injected stale-cache fault and deterministic repeats enforce reproducibility and grading integrity.

19. Supporting artifacts

Files that accompany this submission for task inspection and calibration verification.

Artifact	Contents
covertypes-district-rollout.zip	Complete Harbor task package: instruction, task metadata, environment, workspace, tests, hidden verifier assets and reference solution.
covertypes-district-rollout-traces.zip	Claude Opus 5 calibration rollouts on the shipped build (per-rollout action logs, graded JSON, band verdict and QA review), plus the lower-tier separation set.
Section 13 of this document	Completed quality-control checklist.

20. Acknowledgement

The vendor confirms that the task has been reviewed and is realistic, technically substantive, clearly specified and appropriately graded. It reflects a genuine professional workflow and is not a low-effort or artificial task. The creation and calibration statements above are accurate, and the QA checklist has been completed.
